# The Berkeley Networks of Workstations (NOW) Project

Thomas E. Anderson, David E. Culler, David A. Patterson

Computer Sciences Division/EECS Department

University of California at Berkeley

Berkeley, CA 94720USA

http://now.cs.berkeley.edu

## Abstract

*This paper describes the Berkeley Network of Workstations (NOW) Project, which leverages the high-bandwidth, switch-based local area networks with a custom low-latency network interface and global layer operating system to make a building full of desktop computers act as a single large computer: "The building is the computer."*

## 1 Introduction

Because of recent technology advances, networks of workstations (NOWs) are poised to become the primary computing infrastructure for science and engineering, from low end interactive computing to demanding sequential and parallel applications. We identify three opportunities for NOWs that will benefit end-users: dramatically improving virtual memory and file system performance by using the aggregate DRAM of a NOW as a giant cache for disk; achieving cheap, highly available, and scalable file storage by using redundant arrays of workstation disks, using the LAN as the I/O backplane; and finally, multiple CPUs for parallel computing. We describe the technical challenges in exploiting these opportunities – namely, efficient communication hardware and software, global coordination of multiple workstation operating systems, and enterprise-scale network file systems. We are currently building a 100-node NOW prototype to demonstrate that practical solutions exist to these technical challenges.

---

This paper is a summary of research performed by the Berkeley NOW Project. Information on the project can be found at URL:

http://now.cs.berkeley.edu

The NOW project has received support from the Advanced Research Projects Agency (#F-30602-95-C-0014), the National Science Foundation (CDA-9401156), and the California Micro program. Tom Anderson and David Culler are supported by NSF Presidential Faculty Fellowships. The project has received valued support from SUN Microcomputer Corp., Hewlett-Packard, IBM, Digital Equipment Corp., Intel Corp., Thinking Machines, Synoptics, Cisco, Xerox, AT&T, Siemens, Fujitsu, and Exabyte.

This paper describes the Berkeley NOW project; [1] describes the motivation for NOW as well as the Berkeley NOW Project.

Our approach is guided by a principle of using commercial off-the-shelf systems wherever possible, in recognition of the rapid advance and tremendous investment in such technologies. As a research vehicle, there are two additional advantages: the implementation is quicker if you avoid re-invention and exploiting mostly off-the-shelf technology will simplify the transfer of new ideas and technology.

## 2 Low overhead communication

Bandwidth is the widely advertised metric of communication performance; however, network latency and processor overhead can be just as important, despite their low profile in product literature. This is a peculiar oversight because processor overhead is the dominant factor determining the communication performance of real programs.

Several studies have examined the communication characteristics of parallel programs and many important programs transfer many small messages and are sensitive to communication overhead. What is perhaps more surprising is that many conventional LAN applications exhibit similar characteristics. We obtained a trace of network file system (NFS) traffic over one week from 230 clients of our departmental file servers. Although file transfers are performed in large blocks, we found that 95% of the NFS messages are less than 200 bytes, due to queries to the file system metadata. Moreover, these queries must complete before file data can be returned to the user, so NFS performance is directly coupled to the round-trip message time, i.e., the overhead and latency.

On Sun Sparcstation-10s connected by Ethernet we measure 456 µs of processor overhead plus (unloaded) network latency on a single message and a peak bandwidth of 9 Mb/s through TCP/IP. With the same processors and a Synoptics ATM network the bandwidth increases to 78 Mb/s, but the overhead plus latency also *increases* to 626µs.[1] If we apply these coefficients to our trace, the eight-fold

increase in bandwidth reduces the data transmission time component by that amount, but the overall improvement is just 20% because the overhead plus latency component remains large. This example illustrates that emerging high-bandwidth network technologies will provide a major advance only if they are accompanied by corresponding reductions in latency and processor overhead.

Our target is to perform user-to-user communication of a small message among one hundred processors in 10μs. This is technologically feasible, but leaves very little room for compromise. For example, it is equal to the processor overhead plus network latency on the current CM-5, plus a single serialization delay of an ATM cell. Several aspects of a NOW make us optimistic in meeting this goal, while others make it quite challenging. In NOW we will have faster processors, but greater constraints on where the network connects into the node. We will have somewhat higher link bandwidth, but may have greater routing delay and less than complete reliability. The nodes support a full Unix system, with relatively rigid device and scheduling interfaces.

The focus of our work is on the network interface hardware and the interface into the operating system. To meet our goal, the user must transmit directly into and receive from the network, without operating system intervention. This means that data and control access to the network interface must be mapped into the user address space. The network interface must establish the communication protection domain, which it can do by inserting a network process ID into each outgoing message and checking each incoming message. It also needs the ability to deliver data and notification directly into the user process, at least for the currently running process. If the message is going to awaken a process, other aspects of the notification process will dominate. We do need to buffer messages properly in the meantime, however. Furthermore, the network interface hardware is likely to need to assist in supporting message loss as an infrequent case.

One initial prototype is a cluster of HP9000/735 using an experimental "medusa" FDDI network interface that connects to the graphics bus and provides substantial storage in the network interface. As described in [2], with user level Active Messages we are able to obtain a processor overhead of 8μs, including support of time-out and retry. This also includes almost 3μs of processing that is entirely an FDDI artifact. The network and adapter latency adds an additional 8μs. We are able to obtain the full link bandwidth for large transfers and obtain half of the peak bandwidth on 175 byte messages, compared to 760 bytes for single copy TCP and 1350 for TCP. Constructing conventional sockets on top of this layer, we see a one-way message time of about 25μs, nearly an order of magnitude faster than TCP or single-copy TCP on the same hardware. Our final demonstration system will utilize either a second generation ATM LAN or a retargeted MPP network, such as the Myrinet [3]. We are currently evaluating a spectrum of design alternatives for the network interface card, which will connect either at an emerging high speed external bus, such as PCI, the memory bus, or the graphics bus, depending on our final choice of workstation platform.

The key difference in this work, as compared to traditional LAN interfaces, is first the orientation toward low overhead, low latency communication, second, the quality of the interconnect itself and the simplicity that derives from that, and third, the recognition that we have some control over all the nodes that attach to the network and can thus make strong assertions about the endpoints.

1. Note that these measurements are within 20% of other ATM networks. The network latency component varies for different switches from about 10μs to 100μs depending on the specific configuration. The network interface adapter adds as much as 100μs to the latency, but the largest fraction of the time is the processor overhead resulting from the system software.

## 3 GLUnix: A Global Layer UNIX

The second key challenge is effective management of the pool of resources within a NOW. The idea of globally managing network resources has been around for a long time, yet the most widely used commercial systems do not provide this service. This lack of progress is due in part to two significant impediments: implementing global services in the context of existing commercial operating systems and the sociology of global resource sharing.

### 3.1 GLUnix structure

Recall that the hardware argument for NOWs is that large scale computer systems should be built by networking together small, yet complete, mass-produced commercial systems. The same is true for software. There is a tremendous advantage to leveraging the hundreds of millions of dollars invested each year in commercial operating system development, not to mention the billions invested in application development for these systems. Nevertheless, the typical first step for operating systems research projects is to throw out the commercial system and start from scratch. This is often conceptually easier because it avoids cumbersome artifacts of a working body of code, but building a real working system means re-implementing a huge amount of incidental code (device drivers, virtual memory management, process dispatching, and so on) that already works in commercial systems.

Instead, our approach is to provide the global services of a NOW by "gluing together" local UNIXs running on each workstation on the network[1]. As much as possible,

1. Although we are implementing GLUnix as a layer on top of UNIX, nothing in our approach depends on UNIX as a building block; we could as easily build GLUnix as a layer on top of PC operating systems, such as Windows NT.

this Global Layer UNIX (GLUnix) is built as a layer on top of unmodified commercial UNIXs. By leveraging the complete workstation, including the local operating system, we had a working prototype of GLUnix after only three months effort. This layered approach also better lends itself to tracking advances in the underlying commercial system. The challenge, of course, is performance.

The key technology that allows us to layer efficiently on top of existing systems is software fault isolation [4]. Traditionally, operating system kernels (other than on PC's) use hardware virtual memory to enforce firewalls between user applications. Recent work demonstrates that you can efficiently implement the same firewalls in software, by modifying the application object code to insert a check before every store and indirect branch instruction. By applying aggressive compiler optimization techniques, the overhead of enforcing firewalls in software can be reduced to between 3-7% on several of today's RISC processors. For the same overhead, we can insert a protected *virtual operating system layer* into any UNIX application entirely at user-level; this layer catches and translates the application's system calls, to provide the illusion of a global operating system. For example, we use software fault isolation to implement completely transparent process migration and global resource scheduling.

Of course, it is not possible to provide all the global services a user might want with absolutely no kernel changes. Our goal is to look for the minimal set of changes necessary to make existing commercial systems "NOW-ready". One example of this is replacing the kernel communication software with a low overhead implementation. Another is the use of network RAM within the virtual memory system; this can most easily be implemented by replacing the swap device driver, an operation supported at the user level by some, but not all, modern UNIX systems. As long as the required changes are small, it is feasible to get them included in commercial systems; for example, despite the lack of a compelling market for parallel programs, several years ago industry added synchronization operations (such as "test&set") to processor instruction sets to make their hardware "parallel-ready".

### 3.2 GLUnix sociology

Perhaps the largest roadblock to the success of NOW is the sociology of sharing computing resources. Interactive users look suspiciously at NOW, fearing that demanding applications will steal resources and hurt their interactive response time. After all, one of the principal benefits of the move from timesharing to personal computers a decade ago was the guarantee of a computer to each user. At the same time, supercomputer users also look suspiciously at NOW, fearing that interactive users will have priority and demanding applications will only be allowed to run at night. Like anyone else, supercomputer users work during the daytime, and therefore need good response time even during the daytime [5]. GLUnix needs to address both of these concerns, along with being tolerant of individual node failures. We discuss each of these issues in turn.

We guarantee at least the performance of a stand-alone workstation to every active user, by migrating external processes off an idle machine when the user returns[6].The key to making this approach practical is to consider not only CPU cycles, but memory contents as an interactive resource. On current UNIX systems, if a demanding application runs on your idle workstation, it will eventually flush out your virtual memory pages and file cache contents. When you return to the workstation, your response time will be visibly slowed as your working set is paged back in from disk.[1] Instead, we intend to explicitly save the idle machine's memory contents before using it, so that we can return the machine to the exact state it was in before going idle. This is feasible because of the combination of technologies in NOW; with ATM bandwidth and a parallel file system, 64MB of DRAM can be restored in under 4 seconds. To further reduce complaints by interactive users, we explicitly limit the number of times per day any interactive user can be delayed by external processes.

We also need to deliver a large portion of the aggregate capacity of the system to demanding applications. One issue is that MPP operating systems are typically specialized for scheduling parallel applications, whereas NOWs have independent UNIX kernels on each processor. This local scheduling, employed by parallel environments such as PVM, has the advantage that no system support is required; however, it leads to unacceptable performance for processes that communicate frequently. Our experiments compared the slowdown with local scheduling to co-scheduling all processes of a parallel application[7], as the number of competing parallel jobs increases for five applications.Two of the applications send many small messages to random processors and, as long as enough buffering exists on the destination processor, the sending processor is not significantly slowed. One runs 20 times more slowly even though it communicates infrequently, because it overflows the buffers on the destination. Another suffers from delays encountered at synchronization points to run 40 times more slowly and the final performs 180 times more slowly because processors frequently require data from other processors.

Because many parallel programs run as slowly as their slowest process, parallel performance can also be compromised if one of the machines running the parallel job is also being used for interactive computing. Thus, we need to migrate demanding jobs off no longer idle machine to

1. In fact, at one place where a system was in place to use idle machines for distributed compiles, people in their offices would tap their keyboards periodically simply to keep their memory contents from disappearing!

preserve both interactive *and* parallel performance. Fortunately, our measurements, along with those of others, indicate that a large fraction of workstations are idle, even at the busiest times of the day, so there will usually be a machine to which the evicted process can migrate. For the same reasons, the ability to quickly move processes between machines, along with their memory state, is also important for parallel program performance. While one process is being migrated, the rest of the parallel program is unlikely to be making much progress. By implementing fast process migration and choosing idle machines that are likely to stay idle, a typical P-processor parallel workload can be overlaid on 2P interactive workloads without significantly sacrificing the performance of either. An organization with a more demanding workload would simply have to extend the capacity of its NOW with additional non-interactive machines.

A final consideration in GLUnix is that the system must continue to operate in the face of individual node crashes, new resources being added or deleted from the network, and even operating system software upgrades. On today's multiprocessors, if any CPU fails (or its operating system software crashes), the entire system must be rebooted. Similarly, the entire multiprocessor must be taken out of service to upgrade its hardware or software. This situation makes it impractical to use an MPP or large server as the sole computing infrastructure for a building, since all users would be inconvenienced whenever any small thing goes wrong. Our model is that if a workstation fails, it only affects the programs using that CPU; those programs can be restarted from their last checkpoint, while programs running on other CPUs continue unaffected. We are also structuring our software to tolerate "hot swap" upgrades of hardware and software.

Many consider security to be the Achilles' heel of NOWs. If malicious users can compromise the local operating system on any machine in the NOW, they can corrupt any process or data migrated to that machine. However, many organizations enforce physical security at the level of the entire building, rather than the individual machine. We assume that resource sharing within a NOW will only be used within a single administrative security domain. In addition, a small amount of hardware in the network interface can ensure that the correct operating system is booted on a machine, before allowing it to connect into the NOW. Where tighter security is required, the collection of machines can always be removed from the desktop and placed "in the machine room," with X-terminals on the desktop. But the same basic problems remain no matter where the workstations are physically located. Unlike the mainframes of the past, we must guarantee as good performance as a stand-alone workstation to interactive users by retaining their cached state, and we must provide effective scheduling of parallel applications.

### 3.3 xFS: Serverless Network File Service

Client-server computing has become a popular way of structuring distributed systems. In most network file systems, a central *server* machine provides the abstraction of a single file system shared among the users logged into a number of client workstations. Files are stored on disks at the server, accessible to clients via requests made over the network.

Unfortunately, a central server design has drawbacks in terms of performance, availability, and cost. Any centralized resource will become a bottleneck with enough users. In traditional network file systems, even if clients cache frequently used files, all cache misses and all modified data are sent to the server, ultimately limiting scalability. Furthermore, the DRAM and disk put at the clients do not directly benefit other users. More users can be supported if the file system is partitioned among multiple servers, but this requires the system manager to effectively become *part* of the file system -- moving users, volumes and disks between servers to balance load. Similarly, a central server is a single point of failure, requiring the expense of replicating the server to provide good availability. Perhaps most importantly, server machines are expensive: memory and disks are cheaper in a workstation, even ignoring the cost of server replication for high availability.

In the NOW project, we are addressing these problems by building a completely *serverless* network file system, called xFS[8]. In place of a centralized server (or set of replicated servers), client workstations cooperate in all aspects of the file system – storing data, managing metadata, and enforcing protection. The xFS goal is high performance, highly available network file service that is scalable to an entire enterprise, at low cost.

To achieve this, xFS combines four features not found in other file systems. First, any piece of the file system data, metadata and control can be dynamically migrated between clients and between storage levels; this vastly simplifies both load balancing and failure recovery (any client can take over for any failed client). Second, we use shared-memory multiprocessor-style cache coherence, specifically a write-back ownership protocol, to maximize locality of control and data. Third, we store file data and metadata in a software RAID, a much simpler and cheaper approach to high availability than server replication, at the same time delivering high bandwidth disk I/O to sequential and parallel applications. Finally, we cooperatively manage client caches as a giant cache for disk, and client disk as a giant cache for robotic tape storage, to reduce the I/O bottleneck.

## 4 Conclusion

Computer system design today is dominated by the dramatic rate of advance in small desktop systems, because only these systems offer the large volume and efficiency of production to support a massive on-going investment in architectural innovation. Thus, large scale systems must exploit the desktop system - hardware and software – as a building block, rather than compete with it. The key enabling technology for this "higher order" style of design is a scalable, high bandwidth, low latency network and a low overhead network interface. Coupled with a global operating system layer, the speed of the network allows the vast collection of resources on the network – processors, memories, and disks – to be viewed as shared pool. This view opens up new approaches to traditional system services, including virtual memory, file caching, and disk striping, as well opportunities for large scale parallel computing within an everyday computing infrastructure.

The challenge is to provide the individual user with the fast and predictable response time of a dedicated workstation while allowing tasks that are too large for the desktop to recruit resources throughout the network. The raw performance of the network provides part of the solution, but careful attention must be paid to memory as an interactive resource and to scheduling assumptions in parallel programs. Examination of typical usage characteristics of dedicated workstations and of dedicated MPPs indicate that the two kinds of workloads can be combined in complementary ways, given the ability to detect idle resources and to migrate processes judiciously and quickly. The latter depends critically on a fast network and a parallel file system built out of the workstation disks on that network. By exploiting this confluence of technological advances, we believe NOWs will be the systems of choice for large scale computing within a decade.

## 5 Acknowledgments

The NOW team includes Remzi Arpaci, Satoshi Asami, Tony Chan, Mike Dahlin, Andrea Dusseau, Doug Ghormley, Seth Goldstein, Kim Keeton, Lok Liu, Steve Lumetta, Ken Lutz, Cedric Krumbein, Alan Mainwaring, Rich Martin, Jeanna Neefe, Steve Rodrigues, Drew Roselli, Amin Vahdat, Keith Vetter, Randy Wang, Kristin Wright and Chad Yoshikawa. We are indebted to Terry Lessard-Smith, Bob Miller, and Eric Fraser for terrific administrative and technical support.

## References

[1] T. Anderson, D. Culler, and D. Patterson, "Myrinet – A Gigabit per second Local-area Network," *IEEE Computer*, Vol. 15, No. 1, February 1995.

[2] R. Martin, "HPAM: An Active Message Layer for a Network of HP Workstations," Hot Interconnects II, Aug. 1994.

[3] C. Seitz, "Myrinet – A Gigabit per second Local-area Network," *IEEE Computer*, Vol. 15, No. 1, February 1995.

[4] R. Wahbe, S. Lucco, T. Anderson and S. Graham. "Efficient Software-Based Fault Isolation." *Proc. Fourteenth ACM Symposium on Operating System Principles*, Dec. 1993, pp. 203-216.

[5] R. Arpaci, A. Dusseau, A. Vahdat, T. Anderson, and D. Patterson, "The Interaction of Parallel and Sequential Workloads on a Network of Workstations", CS Technical Report, November 1994.

[6] M. Theimer, K. Landtz, and D. Cheriton, "Preemptable Remote Execution Facilities for the V System," in *Proc. of the 10th ACM Symposium on Operating System Principles*, pp 2-12, Dec 1985.

[7] J. Ousterhout, "Scheduling techniques for concurrent systems", *Proc. 3rd International Conference on Distributed Computing Systems*, pp. 22-30, Oct. 1982.

[8] M. Dahlin, R. Wang, T. Anderson, and D. Patterson, "Cooperative Caching: Using Remote Client Memory to Improve File System Performance," *Proc. of the First Conference on Operating Systems Design and Implementation*, Nov., 1994.